軟體工程課程:
進階軟體測試
(Advanced Software Testing)

海大資工 馬尚彬

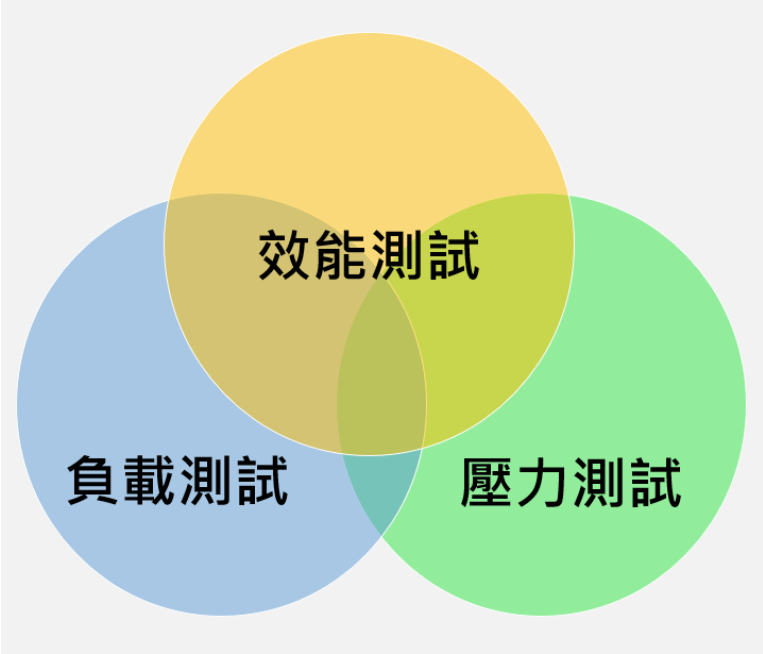
效能/負載/壓力測試

為何需要效能測試？

- 度量系統的反應能力。
- 度量系統的負載能力。
- 了解系統在一般負載(Average Load)與尖峰負載(Peak Load)下之行為。
- 找出系統效能瓶頸。

效能、負載與壓力測試

- 效能測試(Performance Testing)又可分為負載測試(Load Testing)或壓力測試(Stress Testing)等。
  - 一般而言，此三種測試名稱可交替運用，意義相近。
  - 效能測試較為一般化，主要目標為評估非功能需求是否有達成。
  - 負載測試用以評估尖峰負載(Peak Load)之效能。
  - 壓力測試用以評估超載(Overload)情況之效能。



負載之分類

- 平均負載(Average Load)：歷史數據的平均值。
- 尖峰負載(Peak Load)：歷史數據的高點。
- 未來負載(Future Load)：根據目前負載所預測的未來負載，相較於目前系統之可承載壓力，應為超載之情況。

效能測試(Performance Testing)

- 著重在軟體可以處理「多少」資料量(傳輸量)、可以處理「多快」(回應時間)。
- 所有的測試階段都可實施效能測試。
- 要到所有模組完全整合後，才能確定系統的真正效能。
  - 用於驗證系統是否可符合非功能需求(Non-Functional Requirement)。
- 效能測試通常用於多人使用的系統(如：網路系統)，或是具有時間要求性的系統(如：即時系統或嵌入系統)。

負載測試(Load Testing)

- 是一種效能測試。
- 著重於使用預先定義的負載等級來評估系統效能。
- 分析系統在尖峰負載(Peak Load)下的行為，以檢測與負載相關的問題。
  - 大量請求、長時間或高容量(Volume)。
- 根據目標的不同，負載量可以從正常載荷到略高於預期的負載進行實驗，以發現功能或非功能問題。

壓力測試(Stress Testing)

- 是一種效能測試。
- 壓力測試用以評估系統在超出其能力範圍時產生的反應。
- 可逐漸增加負載，了解系統在何種負載情況下無法負荷。
  - 可將負載設定為平均負載之150%~500%。
  - 直到回應時間過慢或錯誤率過高。
- 可針對不同的API進行相同的壓力測試，了解系統瓶頸落於哪些API，以能對症下藥。
  - 同時了解特定API發生故障時，其他功能是否受到影響。

效能/負載/壓力測試的衡量指標

- 使用者的數量 (Number of users)
- 回應時間 (Response time)
  - 使用者需要等待多久時間？
  - 不同使用案例/服務會花費多少時間？
- 錯誤率 (Error Rate)
- 每秒要求數 (Hit per second, Hit/sec)
- 吞吐量 (Throughput)
  - 在一段給定的時間內可以處理多少、多大量的"工作"？
  - 伺服器處理的資料量 (KB/sec)

效能/負載/壓力測試的準則1

1. 分析應用程式，每分鐘可處理多少數量的請求(Request)，每次期望的反應時間(Response Time)為何？
2. 設定正常/最大/過載(Normal/Maximal/Overload)的負載需求值或預期值。
3. 規劃硬體、網路、資料庫、中央處理器(CPU)、記憶體等資源的配置。
4. 以多執行緒(Multi-Thread)模擬多使用者狀態，執行系統資源請求，並進行監控。

效能/負載/壓力測試的準則2

5. 分析測試結果，包括連線數量(Count)、回應時間、錯誤率等。
6. 若不符合測試目標，可適度調整系統配置參數，如：CPU核數、記憶體大小、磁碟(Disk)大小、輸出/輸入內容等相關參數。
7. 完成測試報告。

常見效能/負載/壓力測試工具

- Parasoft SOATest
  - https://www.parasoft.com/products/parasoft-soatest/
- LoadRunner
  - https://www.microfocus.com/zh-tw/products/loadrunner-professional/overview
- Apache JMeter
  - 免費軟體、完全以Java開發。
  - 為功能和效能測試的桌面應用程式
  - 原本僅能測試網頁應用程式，現已支援多種測試標的。
  - https://jmeter.apache.org/

自動化效能測試工具：JMeter

- JMeter下載：http://jmeter.apache.org/download_jmeter.cgi
  - 載binaries (zip檔)，載完後解開 (需先安裝Java 8以上版本)
  - 找bin底下的jmeter.bat (英文)或ApacheJMeter.jar (中文)
- 使用者手冊： http://jmeter.apache.org/usermanual/index.html
- 教學：
  - https://www.blazemeter.com/blog/jmeter-tutorial (英文版)
  - https://openhome.cc/Gossip/JUnit/JMeter.html (中文版)

JMeter簡易使用範例

設定並行之請求數

執行緒群組

名稱 執行緒群組

備註

取樣錯誤後採取的動作

◉ 繼續 ○ Start Next Thread Loop ○ 停止執行緒 ○ 停止測試 ○ Stop Test Now

執行緒屬性

執行緒數量 1000

啟動延遲(秒) 10

迴圈次數： ☐ 永久 10

設定取樣器(Sampler)

HTTP 要求

名稱 HTTP 要求

備註

Basic Advanced

Web 伺服器

協定 https 主機名稱或 IP web-form-2022.herokuapp.com 端口號碼 443

HTTP 要求

POST 路徑 Content encoding:

☐ 自動跟隨重導 ☑ 跟隨重導 ☑ Use KeepAlive ☐ Use multipart/form-data ☐ Browser-compatible headers

Parameters Body Data Files Upload

送出含參數的要求

名稱	值	編...	Content-Type	包含相...
fullname	Ohtani	☐	text/plain	☑
email	ohtani@mlb.com	☐	text/plain	☑

設定監聽器(Listener)以分析結果

彙整報告

名稱 彙整報告

備註

將全部資料寫成檔案

檔名 瀏覽... Log/Display Only: ☐ 只記錄錯誤 ☐ Successes 設定

Label	取樣數	平均值	中間值	90% Line	95% Line	99% Line	最小值	最大值	錯誤率	處理量	每秒仟位元組	Sent KB/sec
HTTP 要求	10000	607	214	1201	2259	8204	197	11503	0.00%	465.2/sec	468.34	115.38
總計	10000	607	214	1201	2259	8204	197	11503	0.00%	465.2/sec	468.34	115.38

JMeter常用監聽器

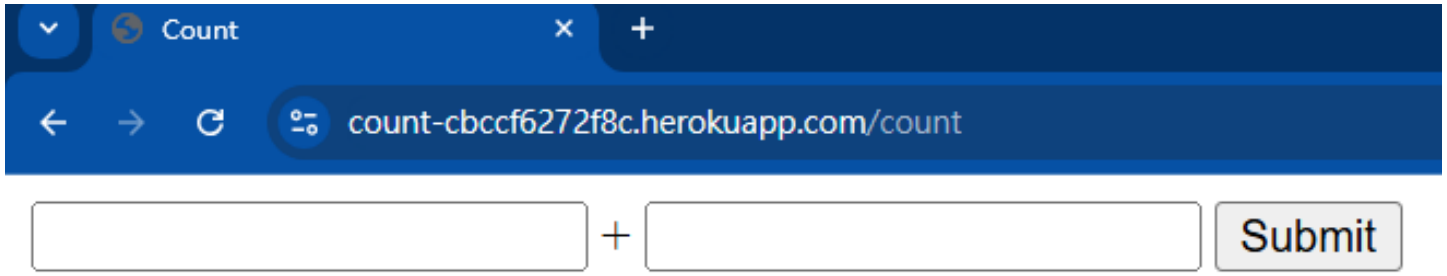
- 檢視結果樹(View Results as Tree):
  - 查看結果樹顯示所有測試樣本的回應，允許使用者檢視任何樣本的請求與回應內容。
- 檢視表格式結果(View Results in Table)：
  - 為每個樣本結果創建一個表格行(Table Row)。
- 彙整報告(Aggregate Report):
  - 為測試中每個不同命名的請求創建一個表格行，呈現彙整資訊。
- 彙整圖形(Aggregate Graph):
  - 表格部分與彙整報告相同。
  - 聚合圖增加了柱狀圖，以呈現各種指標資訊。(後續範例會以此報告為例)

JMeter操作流程

- 我們以”加法”網頁做為受測網頁範例。
  - 需下載範例檔案，並將其解開後放置於tomcat的webapps/ROOT目錄下

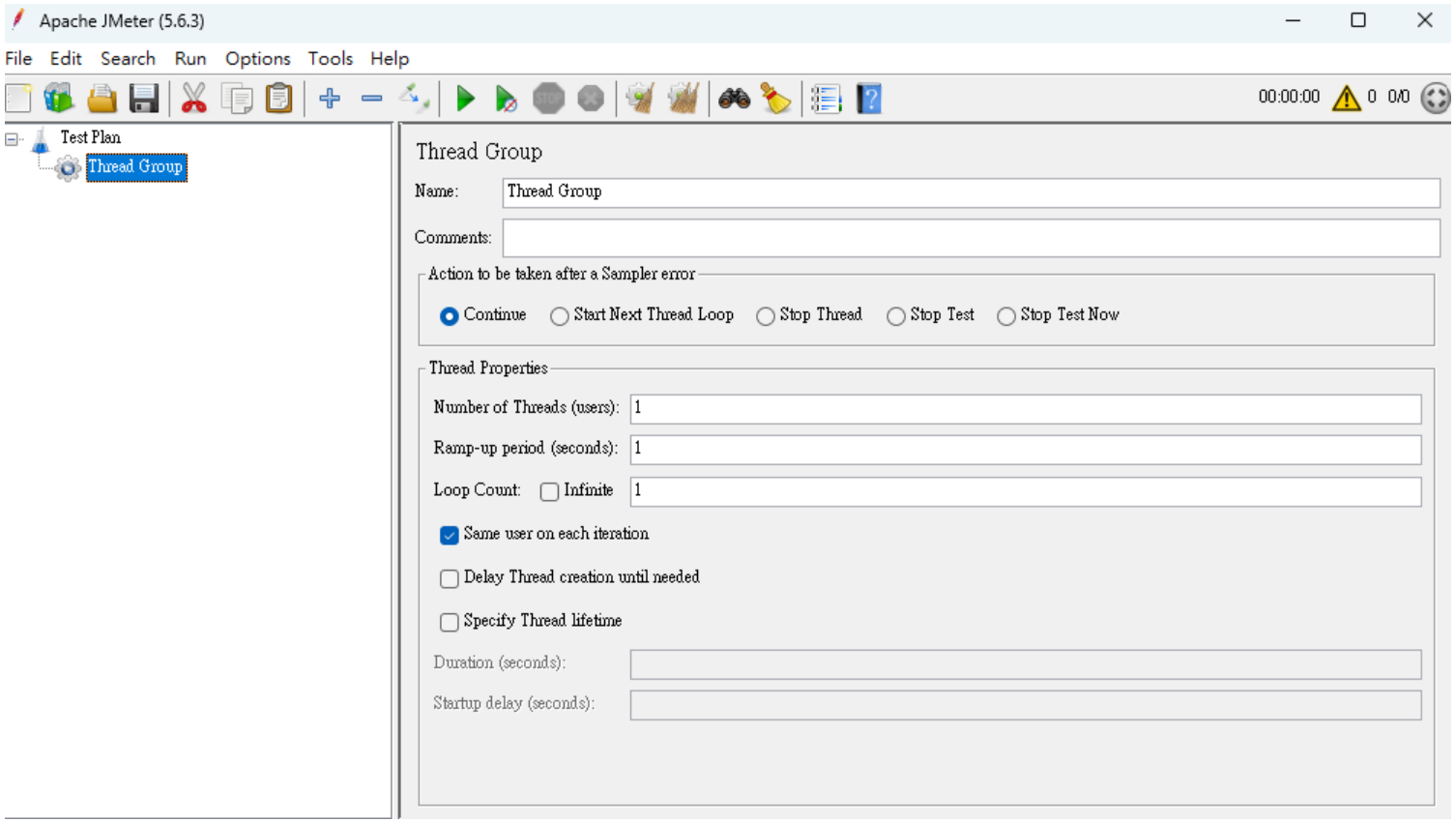
受測網頁範例: 加法程式

- https://count-cbccf6272f8c.herokuapp.com/count



步驟－新增執行緒群組

- 新增執行緒群組並開始新的測試工作
  - 測試計畫右鍵新增Threads(Users): 執行緒群組

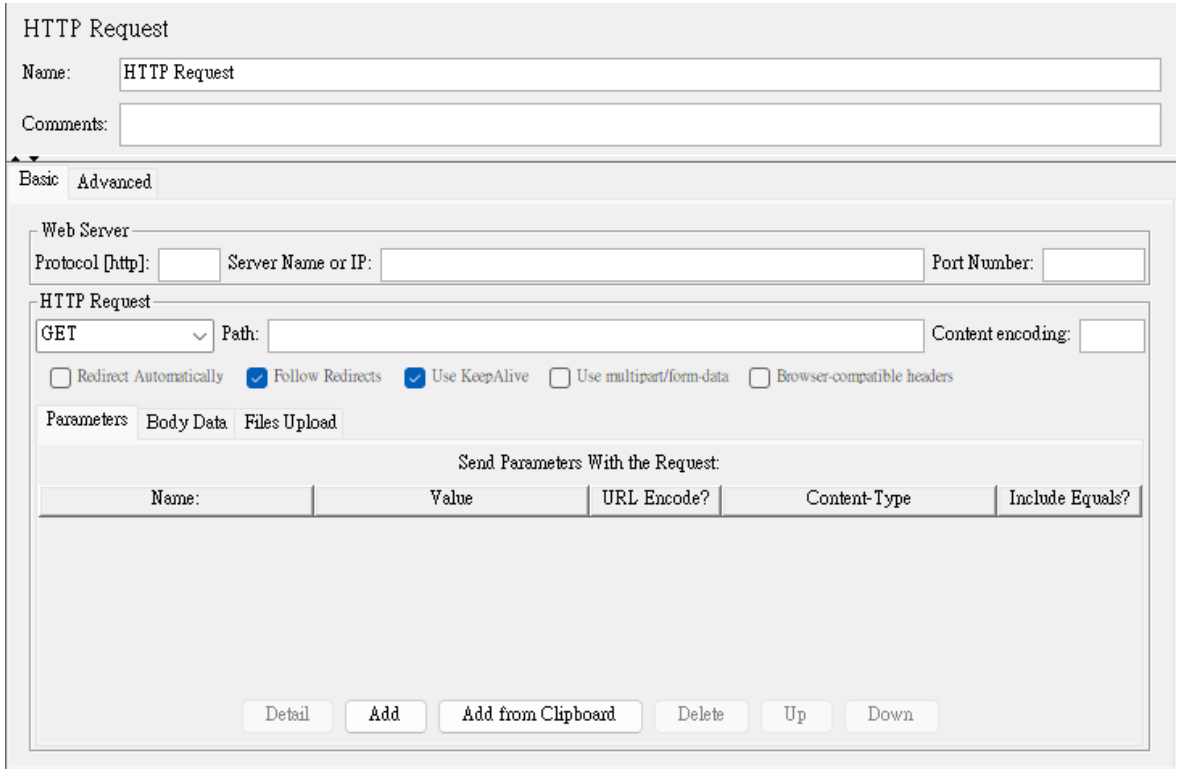


執行緒群組設定

- 若設定執行緒數量為 100，這相當於模擬 100 個使用者對網站的請求。
- 若啟動延遲設定為 10 秒，這表示這 100 個執行緒(使用者)會在 10 秒內陸續請求網站。
- 若迴圈次數設定為 10，表示重複以上情境 10 次，也就是模擬在 100 秒內發出 1000 個 request。

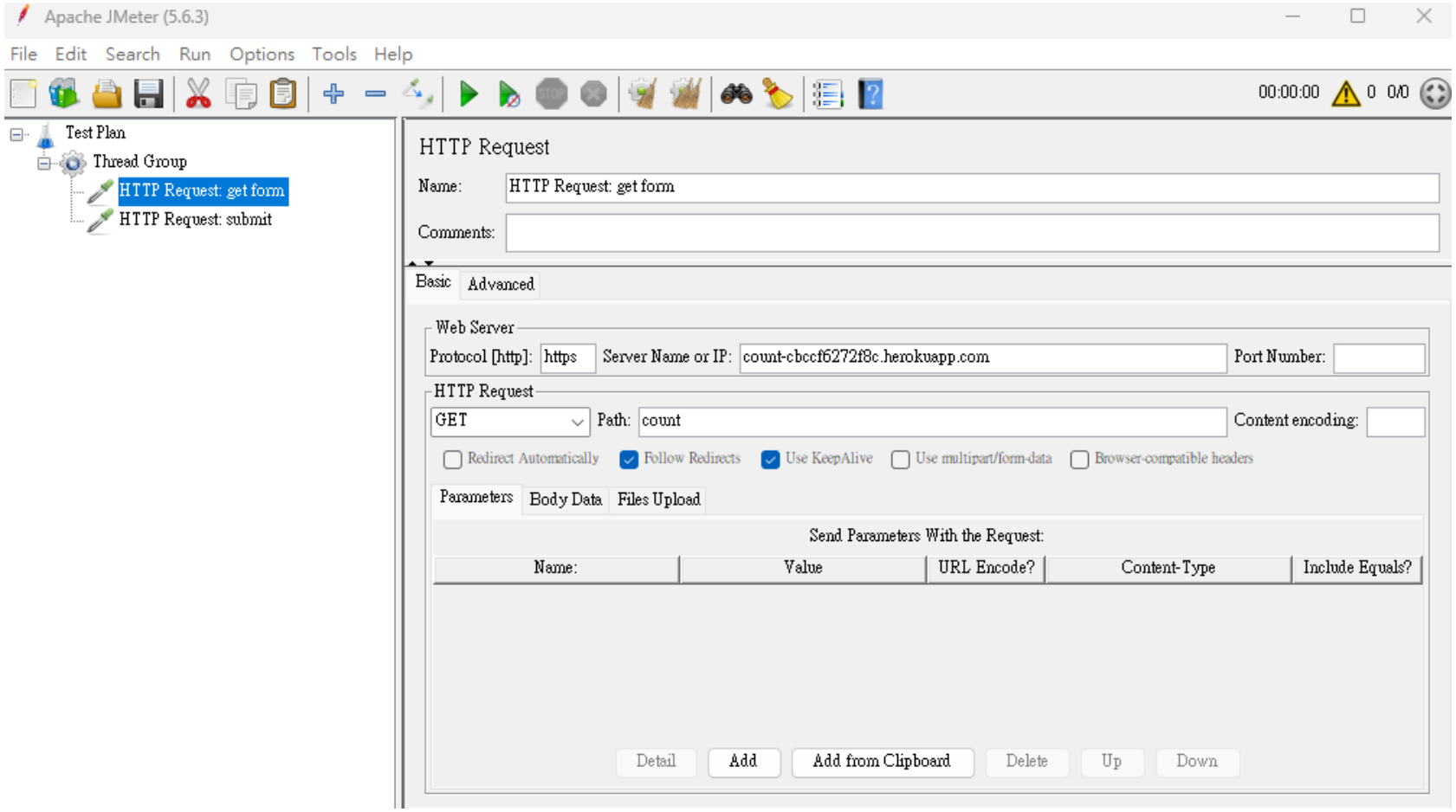
步驟－新增兩個HTTP要求

- 於執行緒群組上點選右鍵，新增[取樣(Sampler)]→[HTTP要求]，並設定測試資料



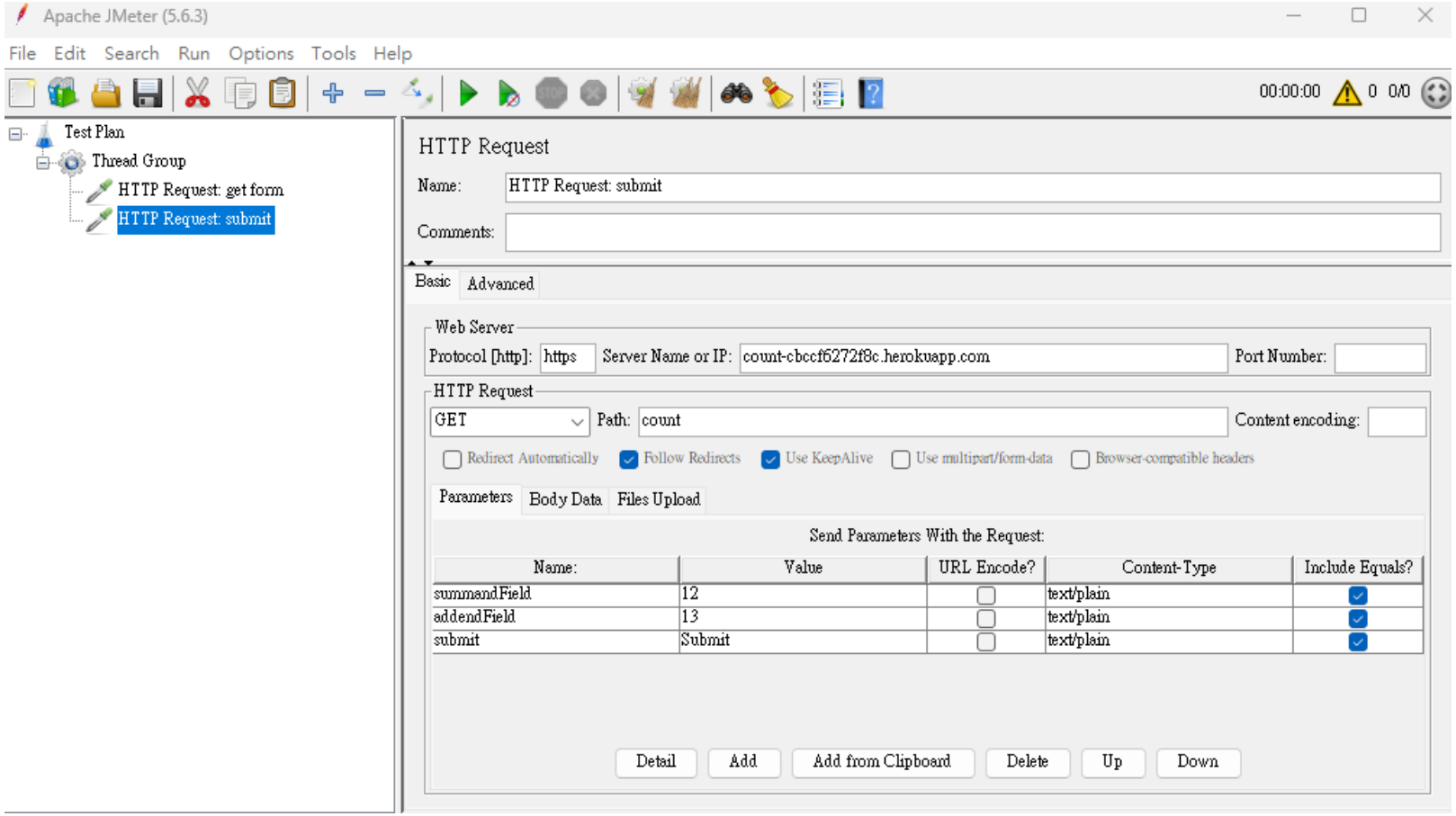
建立第一個HTTP要求 – get form

- IP: count-cbccf6272f8c.herokuapp.com; 路徑: count



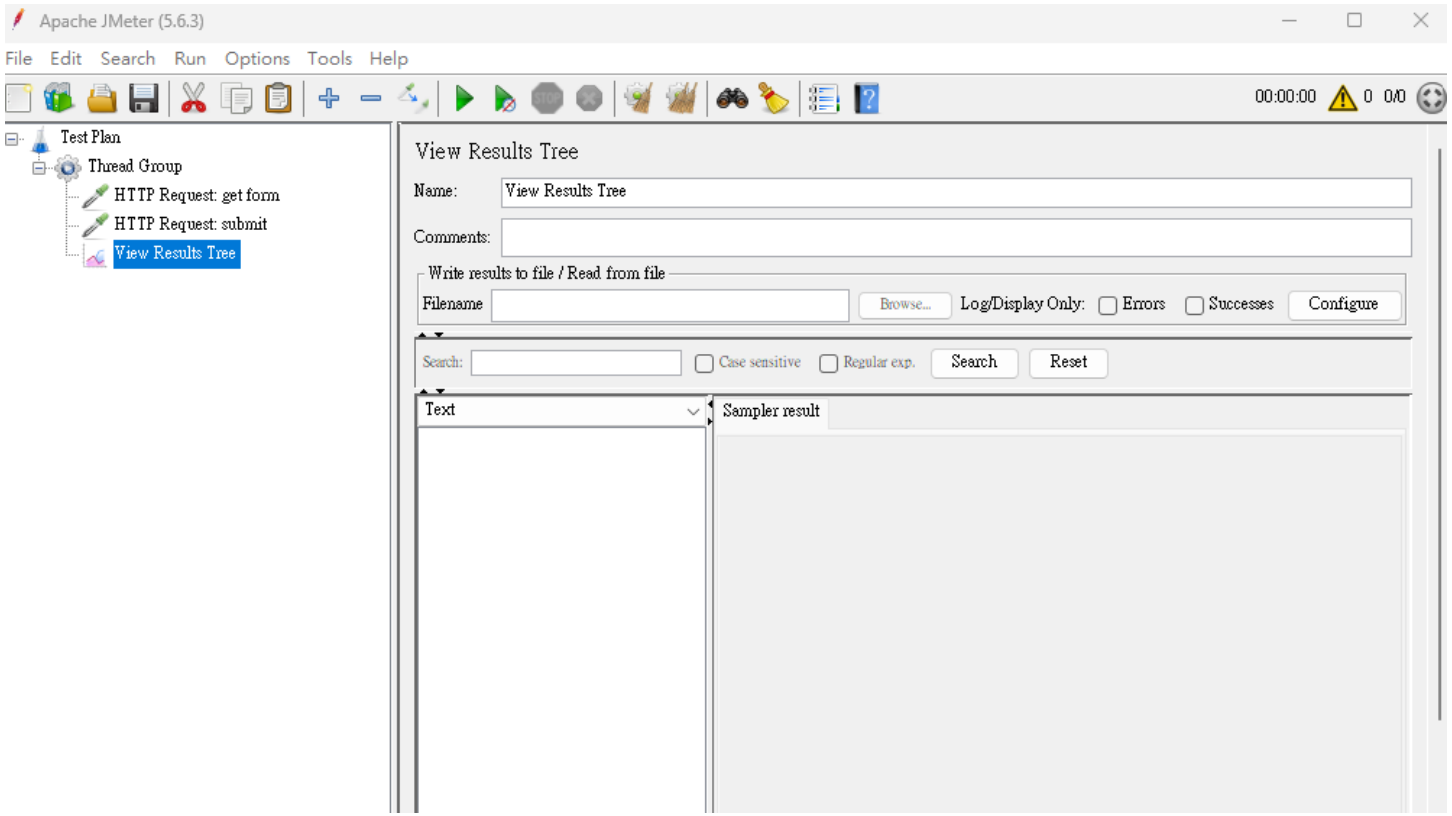
建立第二個HTTP要求

- 在"送出含參數的要求"內新增
  - summandField:12、addendField: 13、submit: Submit



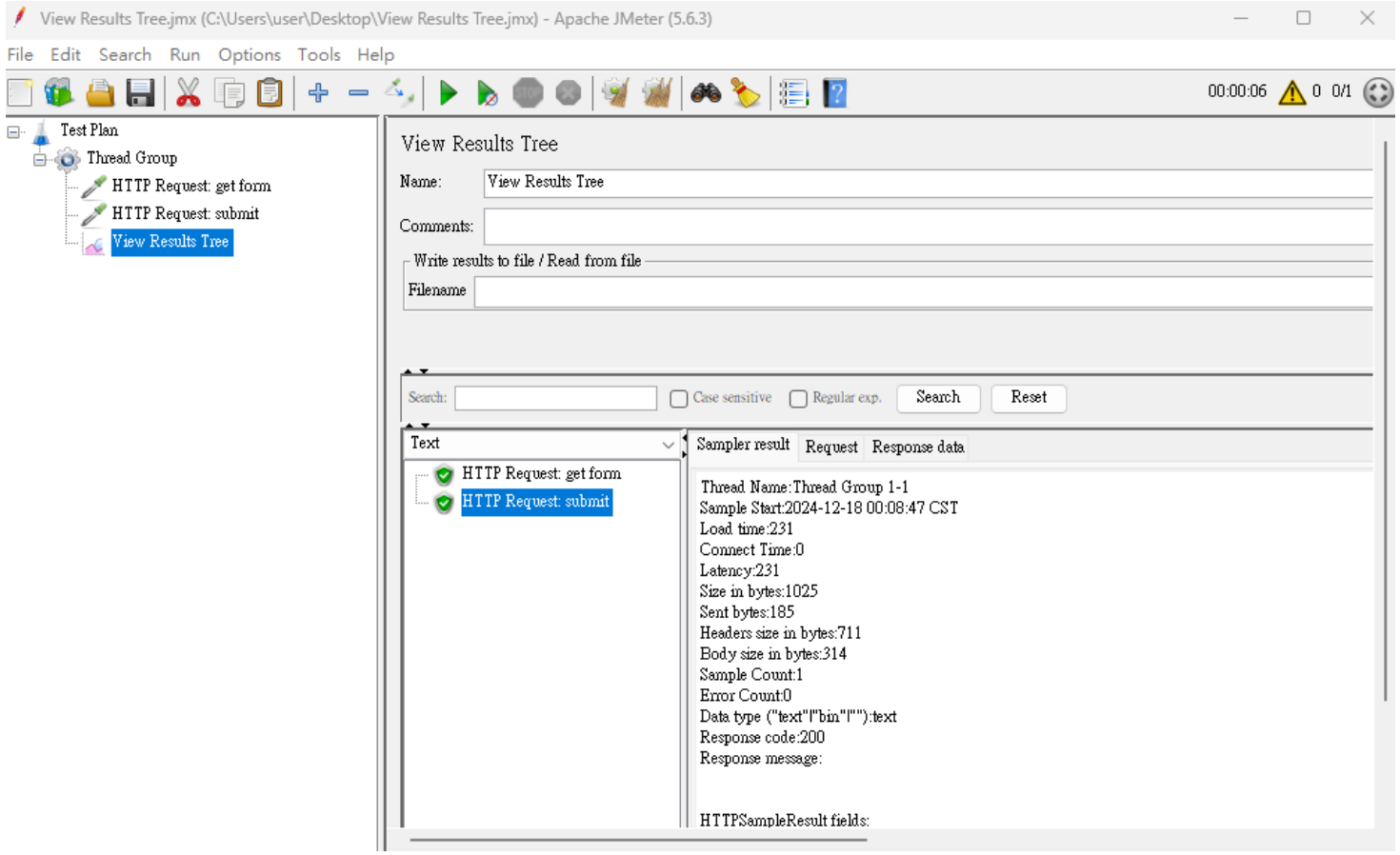
步驟-新增檢視結果樹

- 新增"檢視結果樹(View Results Tree)"以便之後觀察結果
  - 在執行緒群組上點選右鍵，選擇新增→[接聽]→ [檢視結果樹]



步驟-測試結果

- 點選[執行]→[開始]
- 執行結束後，點選檢視結果樹，觀看測試結果



步驟-再新增多種檢視方式

- 新增→[接聽]→[檢視表格式結果]
- 新增→[接聽]→[彙整報告]
- 新增→[接聽]→[彙整圖形]
- 執行測試，觀看結果

可參考 https://octoperf.com/blog/2017/10/19/how-to-analyze-jmeter-results/

JMeter監聽器報告常見指標

- Latency: 度量從發送請求前到收到第一個回應片段的延遲時間。
- Elapsed Time: 度量從發送請求前到接收到最後一個回應片段(response chunk)後的經過時間。
  - 與Latency定義略有不同，可對應至前述之Response Time。
- 中位數 (Median): 將樣本分成相等兩半之中間數值。
- 標準差 (Std. Dev.): 統計上的標準差。
- 9x% Line (90th Percentile): 9x%的樣本均低於此回應時間。
- 錯誤率 (Error Rate): (錯誤請求數÷總請求數)X100
- 處理量 (Throughput): 請求數÷總處理時間

如何分析JMeter指標?

- Latency / Elapsed Time / Connect Time：
  - 數值應盡可能低，建議小於 1 秒。
- 中位數：
  - 應該盡量接近平均回應時間，否則代表回應時間長短不均。
- 9x% Line :
  - 當它低於平均回應時間時，代表部分請求之回應時間明顯高於其他請求，
- 標準差：
  - 應該盡量低。
  - 高標準差代表回應時間存在較大差異，代表有部分請求之回應時間明顯過高。

使用者對回應時間的可接受程度

- 0.1秒：使用者感覺他們正在直接操作使用者介面。
- 1秒：1秒內使用者會認為伺服器正在進行合理的運算。
  - 儘管使用者會留意到短暫的延遲，但他們在1秒鐘的時間間隔內仍然可專注於當前的系統使用流程。
  - 亞馬遜發現每慢100毫秒會損失1%的銷售額(數百萬美元)，
- 10秒：1秒後，使用者開始變得不耐煩，大約 10 秒後，平均注意力持續時間達到最大值。
  - 超過10秒，使用者通常會離開網站，而不是試圖重試系統。

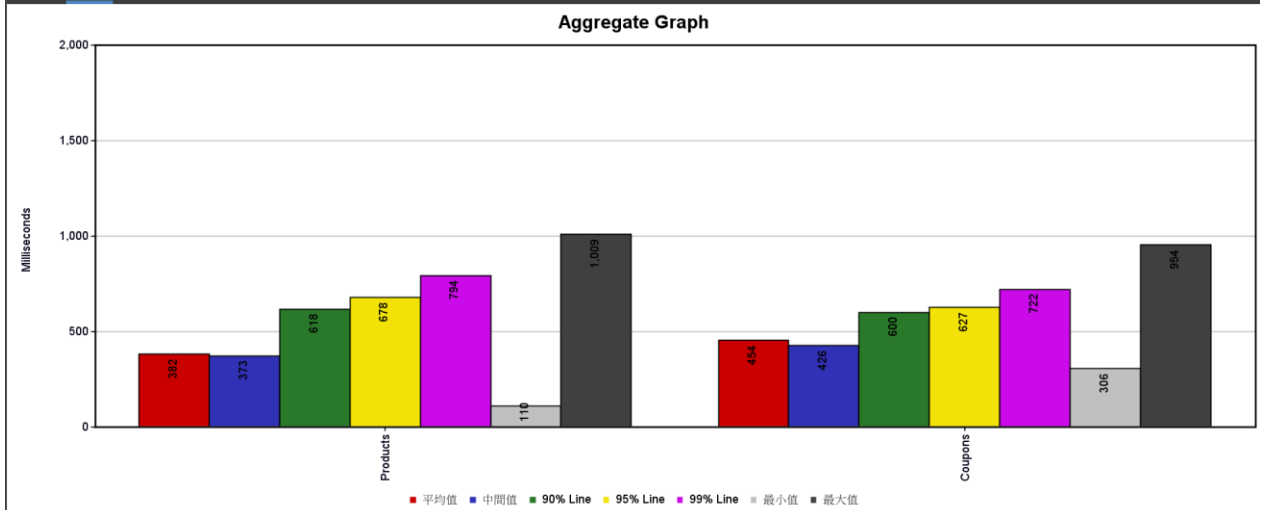
資料來源：https://www.nngroup.com/books/usability-engineering/

簡易案例：商城服務系統1

- 同時的虛擬使用者數為500人。
- 對應之效能需求：
  - 系統應可讓500人同時上線使用。
  - 所有請求應於1秒內回應。
- 此報告顯示，系統實際之效能符合設定之非功能需求。

Label	取樣數	平均值	中間值	90% Line	95% Line	99% Line	最小值	最大值	錯誤率	處理量	每秒仟位元組	Sent KB/sec
Products	500	382	373	618	678	794	110	1009	0.00%	190.0/sec	154.59	23.20
Coupons	500	454	426	600	627	722	306	954	0.00%	177.1/sec	86.28	23.00
總計	1000	418	399	604	649	772	110	1009	0.00%	337.2/sec	219.28	42.47

Settings Graph

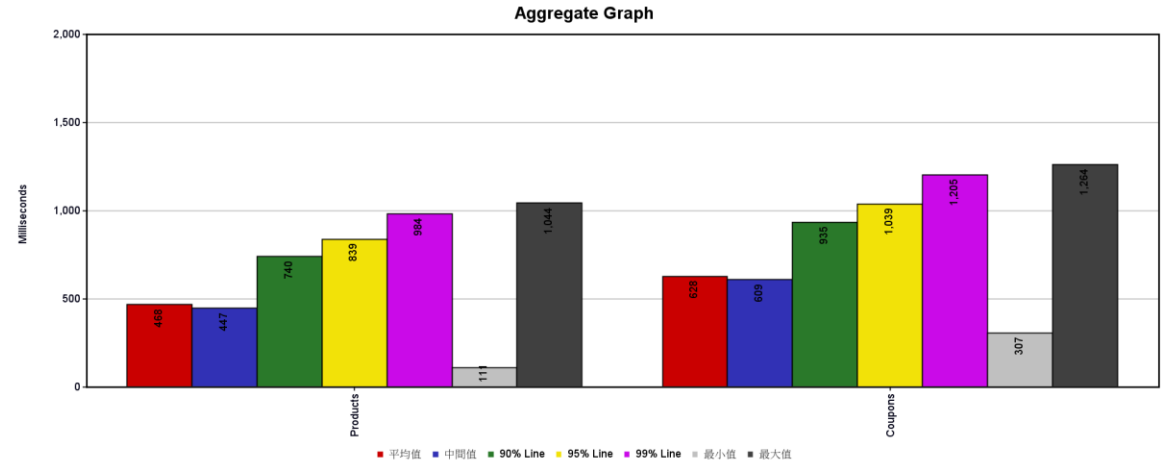


簡易案例：商城服務系統2

- 進行負載測試，確認系統之承載能力。
  - 同時的虛擬使用者數為1000人。(從500持續往上追加)
  - 90%的請求均能於1秒內回傳。
  - 此報告顯示，系統使用者若增倍至1000人，系統仍能順暢運作。

Label	取樣數	平均值	中間值	90% Line	95% Line	99% Line	最小值	最大值	錯誤率	處理量	每秒仟位元組	Sent KB/sec
Products	1000	468	447	740	839	984	111	1044	0.00%	208.3/sec	169.47	25.43
Coupons	1000	628	609	935	1039	1205	307	1264	0.00%	199.1/sec	97.01	25.86
總計	2000	548	532	842	959	1139	111	1264	0.00%	388.6/sec	252.73	48.95

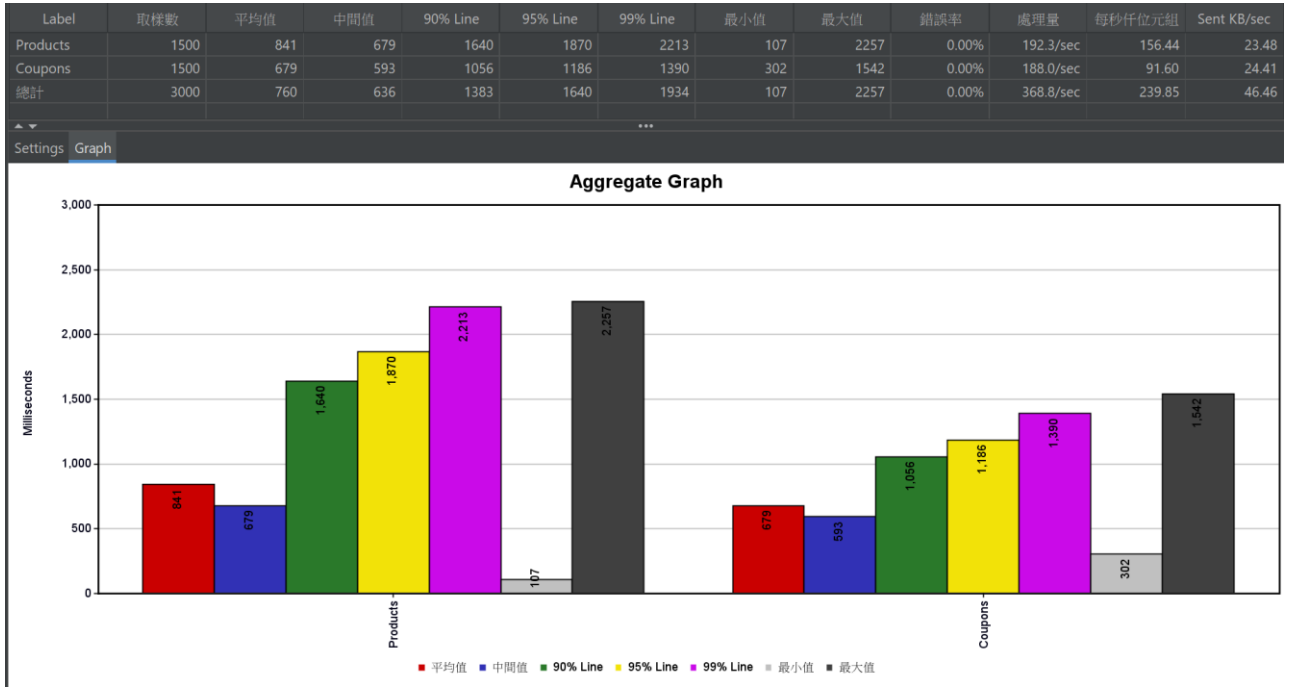
Settings Graph



簡易案例：商城服務系統3

- 進行負載增加之壓力測試。
  - 同時的虛擬使用者數為1500人(之後可一路往上加壓)。
  - 平均回應時間雖能維持於1秒內，但已無法讓90%之請求均能於1秒內完成。
  - 除非擴增資源提升回應速度，否則系統之上限人數建議設定為1000~1500之間。

Label	取樣數	平均值	中間值	90% Line	95% Line	99% Line	最小值	最大值	錯誤率	處理量	每秒仟位元組	Sent KB/sec
Products	1500	841	679	1640	1870	2213	107	2257	0.00%	192.3/sec	156.44	23.48
Coupons	1500	679	593	1056	1186	1390	302	1542	0.00%	188.0/sec	91.60	24.41
總計	3000	760	636	1383	1640	1934	107	2257	0.00%	368.8/sec	239.85	46.46

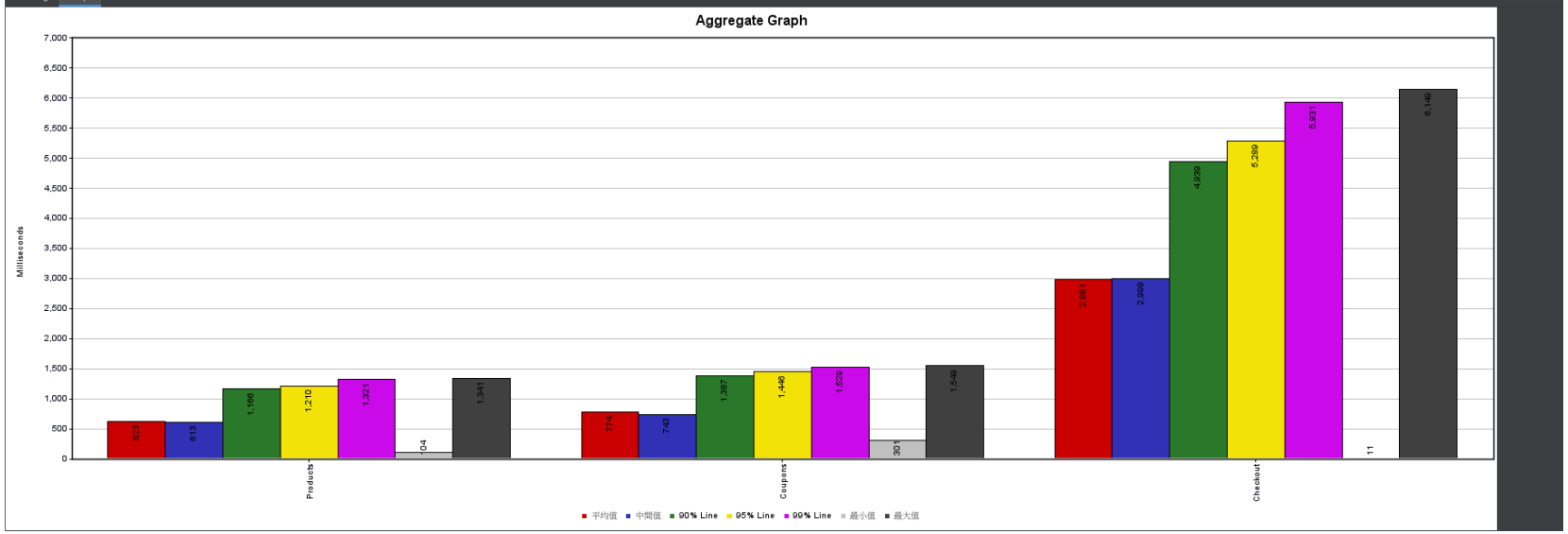


簡易案例：商城服務系統4

- 透過壓力測試找出系統瓶頸點。
  - 同時的虛擬使用者數為1500人。
  - 僅有Checkout功能平均回應時間將近三秒，且有1.2%之錯誤。
  - Checkout元件為可能的系統瓶頸點，應針對其進行效能調校。

Label	取樣數	平均值	中間值	90% Line	95% Line	99% Line	最小值	最大值	錯誤率	處理量	每秒仟位元組	Sent KB/sec
Products	1500	623	613	1166	1210	1321	104	1341	0.00%	76.2/sec	62.01	9.01
Coupons	1500	774	743	1387	1446	1529	301	1549	0.00%	71.7/sec	34.92	9.03
Checkout	1500	2981	2999	4939	5289	5931	11	6149	1.20%	57.7/sec	13.26	7.33
總計	4500	1460	967	3983	4687	5534	11	6149	0.40%	170.3/sec	86.90	21.07

Settings Graph



常見的效能問題來源

- 常見的硬體相關問題
  - 記憶體不足
  - CPU能力不足
  - 網路頻寬不足
- 常見的軟體問題
  - 記憶體執行階段錯誤 (記憶體洩漏或指標錯誤)
  - 資料庫問題 (死結、連線數過多)
  - 多執行緒問題
  - 檔案存取問題

常見的效能提升方法

- 增加運算能力
- 增加記憶體
- 增加儲存空間
- 增加網路頻寬
- 增加平衡負載的能力
- 將大型的圖片檔分割或縮減大小
- 建立快取機制，將常用資料載入在記憶體
- 設定自動延展(Auto-Scaling)

其他測試技術

- 易用性測試(Usability Testing)
  - 易用性準則
  - A/B Testing
- 安全性測試(Security Testing)

易用性測試：易用性準則

使用性測試基準：Nielsen's Usability Heuristics

- NS01 清楚的系統狀態
- NS02 與真實世界的對應
- NS03 使用者擁有控制權
- NS04 一致的風格
- NS05 錯誤預防
- NS06 易於識別
- NS07 有彈性及有效率的使用
- NS08 優雅簡潔的設計
- NS09 清楚的錯誤處理
- NS10 適當的說明

易用性測試：A/B Testing

- A/B Testing 是一種透過分析使用者經驗(UX, User eXperience)來優化介面的方式。
- 進行 A/B Testing 時會設計兩款介面，分為實驗組跟對照組。
  - 相差的變量可以是按鈕顏色、排版設計、圖案等。
  - 當使用者使用網頁時會分別進入兩種不同的版本，而每位使用者在網頁內的動作都會被監控而且Log下來。
  - 根據Log分析，可幫助設計者了解那個版本使用者較滿意。

安全性測試1

- 系統設計者之目標
  - 使進入系統的花費大大超過可獲得資訊的價值
- 預防
  - 必須時常執行資料庫備份。
  - 經常檢查並修補安全性漏洞。
  - 檢查稽核資料庫或系統紀錄(Log)。
  - 稽核資料庫必須保留從上次安全測試至今的資料

安全性測試2

- 測試方法
  - 交由第三方安全測試顧問公司執行測試。
  - 邀請駭客(Hacker)加入測試團隊。
  - 模擬駭客使用各種方法取得密碼，進入破壞系統，或測試未知安全性漏洞，篡改稽核資料庫。
  - 故意造成系統錯誤，利用修復期間進入系統。
  - 透過瀏覽沒有安全保護的資料，找到系統密碼。
  - 偵測非法授權或滲透的系統存取
    - 使用工具監聽所有網路連線埠號(Ftp, Http, SMTP, Pop3 Ports)。
    - 執行應用程式緩衝區過載(Buffer Overflow)檢查。
    - 掃瞄已知安全性漏洞(Holes) ，並分析其風險。

安全性測試3

- 網頁系統測試方法
  - 偵測可疑或惡意連結。
  - HTML內容分析，辨識出內嵌JavaScript或其他的惡意連結。
  - 偵測惡意AJAX執行、跨網站腳本攻擊(XSS, Cross-Site Scripting)、資料隱碼攻擊(SQL Injection)、連線劫持(Session Hijacking) 。
  - 弱點評估(Vulnerability Assessment)與滲透測試(Penetration Testing) 。

Any Question?

